

EECS 2311

LAB 5 (Take Home Assignment)

Mohammad Waliduddin - 219349216

USER STORY	MAIN DEVELOPERS	MEMBER TESTING THE IMPLEMENTATION
Each Tab Functional (ITR1) Allow the user to swap between tabs during their session. These tabs do not have to be fully functional yet but being to swap between them must be. Priority Level: High, Cost: 4 days	Otokpa Attah Pietro Angiolella Rishab Yadav	Mohammad Waliduddin
Add User Authentication (ITR1) Enable users to create accounts, log in, and persist data across sessions. Priority Level: High, Cost: 3 days	Rishab Yadav	Pietro Angiolella
Store User Search History (ITR1) Store the user's search history, allowing users to revisit previous topics they've searched for. Priority Level: High, Cost: 2 days	Rishab Yadav Ondrea Peters	Pietro Angiolella
Display External Resources (ITR1) For each search, display at least 1 external resource or video. Priority Level: High, Cost: 3 days	Mohammad Waliduddin Rishab Yadav	Larissa Singh
Search Course Functional (ITR2) Be able to search a course and for it to grab the Rate Your Prof ratings for the next semester and a quick overview. Priority Level: High, Cost: 5 days	Rishab Yadav Otokpa Attah Larissa Singh Mohammad Waliduddin	Ondrea Peters
Refine Search Topic (ITR2) Refine the search process and roadmap generation. Limit google searches, better search options and be able to click on each Roadmap topic. Priority Level: High, Cost: 3 days	Pietro Angiolella Rishab Yadav	Otokpa Attah
Quiz Generation Possible (ITR2) Have a semi-working quiz generation tab. Will be refined in the coming iterations. Priority Level: Medium, Cost: 3 days	Pietro Angiolella	Rishab Yadav
Refined History (ITR2) Have a working history system with a favouriting system. Priority Level: Medium, Cost: 3 days	Pietro Angiolella Rishab Yadav	Rishab Yadav

5. Next, each student, independently, make a copy of the document and name it EECS2311-SectionName-Team#-YourFullName-TakeHomeAssignment.pdf. In this document, under each assigned story to you, write down several end-2-end manual test cases (equivalent to customer tests) to validate the assigned user stories.
 - a. An end-2-end test is a step-by-step precise instruction to follow a scenario that is described by the user story.
 - b. Make sure your tests are precise enough to be replicable.
 - c. The tests are derived from the user stories from the perspective of customers. Think on different ways they could likely cover the story.
 - d. You need multiple tests per user story so that not only the main scenario is tested but also corner cases.
 - e. Each test is end to end. That is, it uses the GUI, the business layer, as well as all external libraries (if any) and the DB.
6. Continue with writing test and running them until you have a good coverage of the scenarios related to your assigned stories.
7. When you find bugs and issues, report them following the format explained in the lecture on bug reports.
8. Report the bugs both on the team's Github as well as the TakeHomeAssignment document.

The second activity is code review.

This time you are supposed to read the code for the assigned stories. Again, report issues in the Github following the bug report format and also write them down in the document.

- a. You can report both bugs and code/design smells, in this step.

-
- b. Check out the list of code smells here for more information:
 - i. <http://sourcemaking.com/refactoring>
 - ii. <https://refactoring.guru/refactoring/smells>
 - iii. <https://pragmaticways.com/31-code-smells-you-must-know/>
 9. Beside reporting the bugs and issues on the GitHub, write them down in the TakeHomeAssignment document as well. In the document explain what code smells you checked and which ones you detected.

Due:

This activity starts today and ends on March 24 (must be submitted in eClass and GitHub by 23:59).

Deliverable:

Each student submits their “TakeHomeAssignment” document, Individually, on eClass and reports the bugs and issues on the team’s GitHub. The document includes all bugs and issue as they are reported plus the actual test cases and the code smells they checked and those they detected.

Evaluation:

The Rubric and total mark for the assignment will be posted on eClass together with Delivery 2 details.

Task 1: User switch functionality.

Problem Num	Description	Can reproduce	Severity
1	Switching to the Search Course tab displays no results initially, even when valid input is entered. Appears due to tab not loading Firebase/search logic on first open.	Yes	Medium
2	After fixing the Search Course tab (manually refreshing or changing state), switching back to other tabs (e.g., Messaging, Account) becomes non-functional. Tabs appear unresponsive.	Yes	Severe
3	Occasional login redirect issues or complete logout after navigating between tabs. Possibly related to unstable routing or Firebase session timeout during tab switches.	Yes	Severe
4	Inconsistent visual feedback on active tabs. No clear indicator which tab is currently selected. Causes confusion when switching between similar-looking pages.	Yes	Small
5	Application crashes when tabs are rapidly switched multiple times. This is due to multiple Firebase listeners or DOM conflicts being re-initialized without cleanup.	Yes	Medium
6	Pressing the browser's back/forward button doesn't preserve the tab state, and sometimes causes blank pages. No session state is saved.	Yes	Medium

Task 2:

User Story Number	Smell Type	File(s) Involved	Description	Severity
Switch Tabs (ITR1)	Change Preventers	courses.html, account.html, search-topic.html, communities.html, history.html	Tab-switching logic is hardcoded in multiple HTML files, requiring repeated updates. Any changes (like tab highlight, navigation behavior) must be manually replicated across files.	Medium
Switch Tabs (ITR1)	Repetition	courses.js, search-topic.js, communities.js, account.html, history.html	Each JS file implements nearly identical logic for tab switching, active tab styling, and event listeners. These should be extracted into a shared tab-switching utility or module.	Medium
Switch Tabs (ITR2)	Incomplete Code	search-topic.js, google-search.js, courses.js, save-search.js	Tabs like "Search Course" sometimes fail to initialize search or Firebase listeners on first load. Likely due to missing DOMContentLoaded or missing tab activation handling.	Medium
Switch Tabs (ITR2)	Dead Code	courses.js, account.html, generalUI.css	Old tab logic (e.g., inline script tags or redundant tab switch listeners) is left behind after refactoring. Some <script> blocks are inactive or override each other.	Small
Switch Tabs (ITR2)	Repetition	generalUI.css, courses.css, account.css, history.css	Tab styling (.active-tab, .highlight-tab, etc.) is duplicated in each file instead of centralized in generalUI.css. This causes styling conflicts and inconsistencies.	Small
Switch Tabs (ITR3)	Change Preventers	firebaseauth.js, App.js, login.js, status.html, quiz.html, rate-course-experience.html	Tab state is not persisted in routing or session. Users are logged out or redirected on tab switch due to missing auth state checks across Firebase + page reloads.	Severe
Switch Tabs (ITR3)	Incomplete Code	status.html, rate-course-experience.html, quiz.html, generate-roadmap.js	Incomplete or non-working tabs lack full content loading, or certain tabs load blank due to unbound event listeners, null DOM targets, or missing	Medium

			IDs/classes in the HTML structure.	
Switch Tabs (ITR3)	Change Preventers	generate-roadmap.js, generate-test.js, generate-flashcards.js, courses.js	Each generator page defines its own isolated tab navigation, resulting in duplicated logic and inconsistent UX. Breaking changes are hard to scale without shared navigation logic.	Medium
Switch Tabs (ITR1)	Incomplete Code	index.html, App.js	Missing window.history.pushState() or route management for tab memory. Browser back/forward buttons lose state and may show blank or incorrect tab.	Medium